

UNIT – III : INTRODUCTION TO ALGORITHMS

(Full, Detailed, No Topic Missed)

1. What is an Algorithm? (Definition)

Definition:

An **algorithm** is a finite set of step-by-step instructions to solve a problem.

Must-Have Features of Algorithm

Property	Meaning
Input	Must have 0 or more inputs
Output	Must generate at least one output
Definiteness	Each step must be clear, unambiguous
Finiteness	Should stop after finite steps
Effectiveness	Each step must be doable

2. Complexity Concepts

When we evaluate an algorithm, we measure:

a) Time Complexity

“How much *time* it takes as input size grows?”

b) Space Complexity

“How much *extra memory* it uses?”

c) RAM Model

We assume:

- Each simple operation takes **constant time** (addition, comparison, assignment)
-

3. Asymptotic Notations

Used to represent algorithmic growth.

Why? — Visualization

Imagine three functions:

- $f_1(n) = n$
- $f_2(n) = n^2$
- $f_3(n) = \log n$

Plot them in your mind:

- $\log n$ grows VERY slowly
- n grows steadily
- n^2 grows rapidly

Common Notations

Notation	Meaning
$O(f(n))$	Upper bound / worst-case
$\Omega(f(n))$	Lower bound / best-case
$\Theta(f(n))$	Tight bound / exact growth
$o(f(n))$	Strictly smaller order
$\omega(f(n))$	Strictly greater order

Example:

If an algorithm takes $3n^2 + 5n + 2$, the dominating term is n^2

👉 So **time** = $\Theta(n^2)$

4. Recurrences and Their Solutions

A **recurrence** describes running time **in terms of itself**.

Example:

Merge Sort

$$T(n) = 2T(n/2) + O(n)$$

Methods to Solve Recurrences

1. **Substitution Method**
 2. **Recursion Tree Method**
 3. **Master Theorem**
-

5. Design Strategies

A) Recursion

A function calling itself.

Visualization

Imagine a stack of plates:

Each recursive call puts a new plate on the stack.

When base case occurs, plates pop back.

Example: Factorial

Algorithm

```
FACT(n):  
    if n == 0:  
        return 1  
    else:  
        return n * FACT(n-1)
```

C Function

```
int fact(int n) {  
    if (n == 0) return 1;  
    return n * fact(n - 1);  
}
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$ (stack)

B) Divide and Conquer

Break → Solve → Combine.

Examples:

- Merge Sort
- Quick Sort
- Binary Search

Visualization

Breaking chocolate into smaller pieces, solving, then combining.

C) Greedy Method

Pick the **best choice at the moment**, without rethinking.

Visualization

Like taking the **largest coin** first while paying.

Classic Examples

- Activity Selection
 - Fractional Knapsack
-

D) Dynamic Programming

Break problem into **overlapping subproblems**.
Store results to prevent re-computation.

Visualization

Saving answers in a table like “memo notes.”

Classic Examples

- 0/1 Knapsack
- Fibonacci Optimization

6. Sorting Algorithms

We cover:

- ✓ Insertion Sort
 - ✓ Selection Sort
 - ✓ Bubble Sort
 - ✓ Quick Sort
 - ✓ Heap Sort
-

6.1 Insertion Sort

Visualization

Like sorting cards in your hand.

Algorithm

```
for i = 1 to n-1:
    key = A[i]
    j = i-1
    while j >= 0 and A[j] > key:
        A[j+1] = A[j]
        j--
    A[j+1] = key
```

C Code

```
void insertionSort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i];
        int j = i - 1;

        while (j >= 0 && a[j] > key) {
            a[j+1] = a[j];
            j--;
        }
        a[j+1] = key;
    }
}
```

Time Complexity:

- Best: $O(n)$
- Worst: $O(n^2)$
- Average: $O(n^2)$

Space: $O(1)$

6.2 Selection Sort

Visualization

Select the smallest element and swap to front.

C Code

```
void selectionSort(int a[], int n) {
    for (int i = 0; i < n-1; i++) {
        int min = i;
        for (int j = i+1; j < n; j++)
            if (a[j] < a[min])
                min = j;

        int temp = a[i];
        a[i] = a[min];
        a[min] = temp;
    }
}
```

Time: $O(n^2)$

Space: $O(1)$

6.3 Bubble Sort

Visualization

Largest value “bubbles” to top.

C Code

```
void bubbleSort(int a[], int n) {
    for (int i = 0; i < n-1; i++)
        for (int j = 0; j < n-i-1; j++)
```

```

        if (a[j] > a[j+1]) {
            int t = a[j];
            a[j] = a[j+1];
            a[j+1] = t;
        }
    }
}

```

Time: $O(n^2)$

Space: $O(1)$

6.4 Quick Sort (Divide & Conquer)

Visualization

Pick a **pivot**, partition array into:

- left < pivot
- right > pivot

C Code

```

int partition(int a[], int low, int high) {
    int pivot = a[high];
    int i = low - 1;

    for (int j = low; j <= high-1; j++) {
        if (a[j] < pivot) {
            i++;
            int t = a[i]; a[i] = a[j]; a[j] = t;
        }
    }

    int t = a[i+1]; a[i+1] = a[high]; a[high] = t;
    return i+1;
}

void quickSort(int a[], int low, int high) {
    if (low < high) {
        int p = partition(a, low, high);
        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}

```

Time Complexity:

- Best: $O(n \log n)$
- Worst: $O(n^2)$

- Average: $O(n \log n)$

Space: $O(\log n)$

6.5 Heap Sort

Visualization

Build a heap $\rightarrow$ repeatedly remove max.

C Code

```
void heapify(int a[], int n, int i) {
    int largest = i;
    int l = 2*i + 1;
    int r = 2*i + 2;

    if (l < n && a[l] > a[largest]) largest = l;
    if (r < n && a[r] > a[largest]) largest = r;

    if (largest != i) {
        int t = a[i]; a[i] = a[largest]; a[largest] = t;
        heapify(a, n, largest);
    }
}

void heapSort(int a[], int n) {
    for (int i = n/2 - 1; i >= 0; i--)
        heapify(a, n, i);

    for (int i = n - 1; i > 0; i--) {
        int t = a[0]; a[0] = a[i]; a[i] = t;
        heapify(a, i, 0);
    }
}
```

Time: $O(n \log n)$

Space: $O(1)$

7. Searching Algorithms

We cover:

- ✓ Linear Search
 - ✓ Binary Search
 - ✓ Selection in Linear Time (Median of Medians)
-

7.1 Linear Search

Visualization

Check every element.

C Code

```
int linearSearch(int a[], int n, int key) {
    for (int i = 0; i < n; i++)
        if (a[i] == key)
            return i;
    return -1;
}
```

Time: $O(n)$

Space: $O(1)$

7.2 Binary Search (Divide & Conquer)

Visualization

Keep halving the sorted array.

C Code

```
int binarySearch(int a[], int n, int key) {
    int low = 0, high = n-1;

    while (low <= high) {
        int mid = (low + high)/2;

        if (a[mid] == key) return mid;
        else if (key < a[mid]) high = mid-1;
        else low = mid+1;
    }
    return -1;
}
```

}

Time

- $O(\log n)$

Space

- $O(1)$
-

7.3 Selection in Linear Time

(Finding k-th smallest element)

Uses **Median of Medians** algorithm.

Complexity

- Worst Case: **$O(n)$**
- Space: $O(1)$